

Count Ways to Write n as Sum of Positive Integers (Integer Partition)

Problem Statement

Given a positive integer n , find the number of different ways in which n can be written as a sum of **two or more** positive integers.

Return the answer modulo $10^9 + 7$.

Example

```
n = 4
```

```
Ways:
```

```
3 + 1
```

```
2 + 2
```

```
2 + 1 + 1
```

```
1 + 1 + 1 + 1
```

```
Answer = 4
```

Step 1: Understand the Problem

The important word is:

```
different ways
```

This means:

```
1 + 3 = 3 + 1
```

They are considered the **same**.

So:

- Order DOES NOT matter.
- Numbers CAN be reused.

This is an **Integer Partition** problem and follows the **Coin Change II** pattern.

Step 2: Brute Force Thinking

For $n = 4$, we can generate:

```
1+1+1+1
1+1+2
1+3
2+2
4
```

Total = 5.

Since the question wants:

sum of two or more integers

remove:

```
4
```

Final answer = 4.

Step 3: Why `solve(rem)` is NOT enough

Suppose I ask:

```
Make sum = 5
```

Is this enough information?

No.

Because:

```
Make 5 using {1,2,3,...}
```

and

```
Make 5 using {2,3,4,...}
```

have different answers.

We need another piece of information:

Which numbers are allowed?

Step 4: Build the State

Define:

```
solve(num, rem)
```

Meaning:

Number of ways to make `rem` using numbers `num`, `num+1`, `num+2`, ...

Step 5: Choices

At every number `num`, we have only two choices:

Choice 1: Take `num`

Example:

```
solve(2,5)

Take 2:

2 + ?
remaining = 3
```

New state:

```
solve(2,3)
```

Notice:

We stay at `2` because we can reuse it.

Choice 2: Skip **num**

Don't use **2**.

Move to next number.

New state:

```
solve(3,5)
```

Recurrence Relation

```
solve(num, rem)  
=  
solve(num, rem-num)  
+  
solve(num+1, rem)
```

Step 6: Base Cases

Base Case 1

If:

```
remaining sum = 0
```

we found one valid way.

```
if rem == 0:  
    return 1
```

Base Case 2

If:

```
num > rem
```

Example:

```
Make 3 using {5,6,7,...}
```

Impossible.

```
if num > rem:  
    return 0
```

Recursive Solution

```
def solve(num, rem):  
    if rem == 0:  
        return 1  
  
    if num > rem:  
        return 0  
  
    take = solve(num, rem - num)  
    skip = solve(num + 1, rem)  
  
    return take + skip
```

Final answer:

```
solve(1, n) - 1
```

Subtract 1 because:

```
n = n
```

is also counted.

Memoization

Rule

Memo key = DP state.

State:

```
solve(num, rem)
```

Memo key:

```
(num, rem)
```

Memoized Code

```
MOD = 10**9 + 7

def countWays(n):
    memo = {}

    def solve(num, rem):
        if rem == 0:
            return 1

        if num > rem:
            return 0

        if (num, rem) in memo:
            return memo[(num, rem)]

        take = solve(num, rem - num)
        skip = solve(num + 1, rem)

        memo[(num, rem)] = (take + skip) % MOD

        return memo[(num, rem)]

    return (solve(1, n) - 1) % MOD
```

Converting Memoization → Tabulation

This is the MOST IMPORTANT PART.

Step 1: Table Dimensions

State:

```
solve(num, rem)
```

Two variables.

Therefore:

```
dp[num][rem]
```

2D DP table.

Rule

State	DP Table
solve(i)	dp[i]
solve(i,j)	dp[i][j]
solve(i,j,k)	dp[i][j][k]

Step 2: Meaning of DP Table

Always write:

```
dp[num][rem]
```

means:

Number of ways to make **rem** using numbers **num, num+1, ...**

This sentence is extremely important.

Step 3: Table Size

```
num : 1 to n+1  
rem : 0 to n
```

Therefore:

```
dp = [[0]*(n+1) for _ in range(n+2)]
```

Step 4: Convert Base Cases

Memo:

```
if rem == 0:  
    return 1
```

Tabulation:

```
for num in range(n+2):  
    dp[num][0] = 1
```

Memo:

```
if num > rem:  
    return 0
```

No need to initialize.

DP table already contains zeros.

Step 5: Convert Recurrence

Memo:

```
take = solve(num, rem-num)  
skip = solve(num+1, rem)  
  
answer = take + skip
```

Tabulation:


```
take = dp[num][rem-num]
skip = dp[num+1][rem]

dp[num][rem] = take + skip
```

Step 6: Determine Filling Order

Look at dependencies.

We need:

```
dp[num][rem-num]
dp[num+1][rem]
```

Dependency 1

```
dp[num][rem-num]
```

Same row.

Smaller column.

Therefore:

```
left → right
```

Dependency 2

```
dp[num+1][rem]
```

Next row.

Therefore:

```
bottom → top
```

Final Traversal

```
for num in range(n, 0, -1):
    for rem in range(1, n+1):
```

Final DP Code (Fully Commented)

```
MOD = 10**9 + 7

def countWays(n):

    # dp[num][rem]
    # = ways to make rem using numbers num,num+1,...

    dp = [[0] * (n + 1) for _ in range(n + 2)]

    # Base Case:
    # If remaining sum is 0,
    # we found one valid way.
    for num in range(n + 2):
        dp[num][0] = 1

    # Fill rows from bottom to top
    for num in range(n, 0, -1):

        # Fill columns from left to right
        for rem in range(1, n + 1):

            # Don't take current number
            skip = dp[num + 1][rem]

            # Take current number
            take = 0
            if rem >= num:
                take = dp[num][rem - num]

            # Total ways
            dp[num][rem] = (take + skip) % MOD

    # Remove partition containing only n itself
    return (dp[1][n] - 1) % MOD
```

Universal Memo → Tabulation Checklist

Step 1

Write:

```
dp[i][j] means ?
```

in English.

Step 2

Copy base cases.

Step 3

Replace:

```
solve(...)
```

with

```
dp[...]
```

Step 4

Study dependencies.

Dependency Rules

If:

```
dp[i][j]  
depends on  
dp[i-1][j]
```

Fill:

```
top → bottom
```

If:

```
dp[i][j]  
depends on  
dp[i+1][j]
```

Fill:

```
bottom → top
```

If:

```
dp[i][j]  
depends on  
dp[i][j-1]
```

Fill:

```
left → right
```

If:

```
dp[i][j]  
depends on  
dp[i][j+1]
```

Fill:

```
right → left
```

How to Build States in DP

Ask:

Question 1

What choices do I have?

Question 2

After making a choice, what information do I need to continue?

That information becomes the state.

Examples

Climbing Stairs

Choices:

```
Take 1 step  
Take 2 steps
```

State:

```
solve(n)
```

LCS

Need:

```
Position in string1  
Position in string2
```

State:

```
solve(i,j)
```

Knapsack

Need:

```
Current item
```

```
Remaining capacity
```

State:

```
solve(i, cap)
```

Coin Change II / Integer Partition

Need:

```
Current number  
Remaining sum
```

State:

```
solve(num, rem)
```

Pattern Recognition

Whenever you see:

```
Count ways  
Order doesn't matter  
Elements can be reused
```

Think:

```
Coin Change II Pattern
```

State:

```
solve(current_number, remaining_sum)
```

Choices:

Take
Skip

Recurrence:

```
solve(num, rem)
=
solve(num, rem-num)
+
solve(num+1, rem)
```

Similar Problems

1. Coin Change II
 2. Integer Partition
 3. Perfect Sum Problem
 4. Target Sum
 5. Count of Subsets with Given Sum
 6. Rod Cutting
 7. Unbounded Knapsack
 8. Minimum Coins
 9. Combination Sum IV (order matters version)
 10. Partition Equal Subset Sum
-

Biggest Lesson From This Problem

```
Recursion → Choices
State → Information needed to continue
Memo key → State itself
DP dimensions → Number of state variables
Table filling order → Dependencies
```

This is the complete workflow for converting:

Problem

- Recursion
- Memoization
- Tabulation
- Space Optimization